

Query 17 and 8:

Q17 — Stage-by-Stage Analysis (RLS vs View)

Times: RLS = 5,170,421 ms (~86 min) | View = 4,887,693 ms (~81 min)

Policy / View Filter Applied

RLS: `lineitem_rls_policy` — only rows where `EXISTS (SELECT 1 FROM supplier s WHERE s.s_suppkey = lineitem.l_suppkey AND s.s_acctbal > 1000)`

View: `lineitem_secure` — same logic, `WHERE EXISTS (SELECT 1 FROM supplier s WHERE s.s_suppkey = l.l_suppkey AND s.s_acctbal > 1000)`

Same logical filter, but the execution machinery is completely different.

Stage 1 — Outer Lineitem Scan

RLS:

```
Seq Scan on public.lineitem
cost=0.00..2,208,503,289.20   actual time=3.668..1573ms   row
s=4,918,316
```

The cost is astronomically inflated — `2.2 billion` — because PostgreSQL **cannot push any filter through the security barrier**. It must evaluate the RLS `EXISTS` check for every single one of the 6 million rows in lineitem before applying any other predicate. The `s_acctbal > 1000` check joins to supplier for each row individually.

View:

```
Hash Semi Join (lineitem l ⋈ supplier s on l_suppkey = s_sup
pkey)
cost=445.45..235,643.75   actual time=11.338..1929ms   rows=
4,918,316
```

The planner sees the view as a transparent subquery and **flattens it into the main plan**. It builds a hash of the ~8,197 qualifying supplier keys once, then streams lineitem through a Hash Semi Join. Cost drops from `2.2 billion` → `235,643`. This is the single biggest difference in the entire query.

Key difference at this stage: RLS barrier = row-by-row correlated EXISTS. View = one-time hash build + single pass join. The view scan is ~9,370x cheaper in estimated cost.

Stage 2 — Correlated Subquery (Inner `0.7 * avg(l_quantity)`)

This subquery runs **5,347 times** (once per distinct `p_partkey` from part) in both plans.

RLS inner loop:

```
Aggregate (cost=2,208,518,291.54..56 per loop)
  Seq Scan on lineitem lineitem_1
    cost=0.00..2,208,518,291.50    actual time=41.873..966ms per
    loop
```

Again the barrier forces a full lineitem scan per iteration, inflated by the supplier EXISTS check. Each of the 5,347 iterations scans ~6M rows against the supplier join. Total inner cost = `5,347 × 966ms ≈ 5,165 seconds` — **this is where all the time goes**.

View inner loop:

```
Aggregate → Hash Semi Join (cost=180,177.37 per loop)
  Seq Scan on lineitem l_1
    cost=0.00..179,731.50    actual time=30.628..913ms per loop
```

The view's supplier filter is again flattened into a Hash Semi Join inside the subquery. Cost per loop drops to `180,177` vs `2,208,518,291`. But actual time per loop is still ~913ms — because the hash needs to be rebuilt each iteration (supplier hash is re-used from cache but lineitem still scans fully). Total = `5,347 × 913ms ≈ 4,882 seconds`.

Key difference at this stage: Estimated cost per loop is 12,000x lower for View. Actual time difference is only ~8% (966ms vs 913ms per loop) because both still do a full lineitem scan — the view just avoids the correlated EXISTS overhead per row.

Stage 3 — Part Table Scan

Both: Almost identical — `Parallel Seq Scan on part` filtered on `p_brand='Brand#53' AND p_container='MED BAG'`, producing 214 rows. Negligible difference. RLS: 21ms, View: 146ms (slightly slower due to parallel overhead in that run).

Stage 4 — Hash Join (lineitem ⋈ part)

RLS:

```
Hash Join cost=6,136.16..6,528,380,072,259
actual time=2,279ms..5,170,416ms
```

The outer scan's inflated cost cascades here. The join itself is fine (hash on 214 part keys) but is dominated by the time spent waiting for the correlated subquery to evaluate each row.

View:

```
Hash Join cost=6,581.61..873,219,724
actual time=1,432ms..4,887,688ms
```

Same join strategy, lower cost. The ~5 min savings vs RLS comes entirely from the inner loop being slightly faster per iteration.

Stage 5 — Final Aggregate

RLS: `Aggregate actual time=5,170,419..5,170,420ms` (essentially the entire query time)

View: `Aggregate actual time=4,887,690..4,887,691ms`

Both are trivial single-row aggregates — the time shown is just when the last row finally arrived from below.

Summary Table — Q17

Stage	RLS Cost	View Cost	Actual RLS	Actual View
Outer lineitem scan	2,208,503,289	235,643	1,573ms	1,929ms
Inner corr. subquery ×5347	2,208,518,291 each	180,177 each	~966ms/iter	~913ms/iter
Part scan	~6,133	~6,133	21ms	146ms
Hash join	6.5 trillion est	873 billion est	dominates	dominates
Total	—	—	5,170,421ms	4,887,693ms

Why are they both slow? The correlated subquery `0.7 * avg(l_quantity)` runs 5,347 times either way, and each iteration requires a full lineitem scan. The view saves ~5 min by eliminating the per-row EXISTS overhead in the outer scan, but can't eliminate the fundamental $O(n^2)$ correlated subquery problem.

Q17 has a correlated subquery that runs once per matching part key (~5,347 times), and each time it needs to scan the entire lineitem table (~6M rows) to compute an average. Both RLS and View suffer from this equally – neither can escape the fundamental $O(n^2)$ problem. The only difference is that RLS adds a small extra cost per row in the outer scan because the security barrier forces a correlated EXISTS check against supplier for every lineitem row individually, while the view merges that supplier check into a clean Hash Semi Join upfront. So RLS takes 86 min and View takes 81 min – both are slow for the exact same root reason (the correlated subquery), and the 5 minute gap is just the small overhead RLS adds on top.

Q8 — Why View is Extremely Fast (825ms vs RLS 745,310ms)

Times: RLS = 745,310ms (~12.4 min) | View = 825ms | Baseline = 874ms

The view is nearly as fast as running with no filter at all. This is one of the most dramatic differences possible between RLS and views, and the reason is architectural.

The Policy Applied

RLS on orders:

sql

```
CREATE POLICY orders_rls_policy ON orders
USING (
  EXISTS (
    SELECT 1 FROM customer c
    JOIN nation n1 ON c.c_nationkey = n1.n_nationkey
    JOIN region r ON n1.n_regionkey = r.r_regionkey
    WHERE c.c_custkey = orders.o_custkey
    AND r.r_name = 'ASIA'
  )
);
```

View:

sql

```
CREATE VIEW orders_secure AS
SELECT o.*
FROM orders o
WHERE EXISTS (
  SELECT 1 FROM customer c
  JOIN nation n1 ON c.c_nationkey = n1.n_nationkey
  JOIN region r ON n1.n_regionkey = r.r_regionkey
  WHERE c.c_custkey = o.o_custkey
  AND r.r_name = 'ASIA'
);
```

Identical logical definition. Completely different planning outcome.

Factor 1 — Security Barrier Prevents Join Reordering (Root Cause)

This is the core reason. PostgreSQL marks RLS policies as **security barriers**, meaning the planner is **forbidden** from pushing outer query predicates through them — it cannot reorder joins, push down filters, or merge the policy subquery with the main query's join tree.

The result in the RLS plan:

```
Seq Scan on public.orders
cost=0.00..8,394,198,508.00  actual time=55ms..1,258ms  rows=91,707
```

Cost of **8.39 billion** for a table that has 1.5M rows. Why? Because for every row in orders, PostgreSQL must evaluate the EXISTS subquery — join to customer (150k rows), nation (25 rows), region (5 rows) — as a **correlated check per row**. The planner cannot batch this or reorder it.

The view planner faces no such restriction and instead builds this:

```
Parallel Hash Semi Join on o.o_custkey = c.c_custkey
  → Parallel Seq Scan on orders o  cost=35,383  actual time=0.686..94ms
```

Cost drops **from 8.39 billion** → **35,383**. The **view** query's order scan costs essentially the same **as** the baseline unfiltered query.

Factor 2 — View Allows Full Join Flattening

In the view plan, the entire query — including the view's customer/nation/region filter — is compiled into a single unified join tree:

```
GroupAggregate
  → Sort
    → Hash Join (supplier ⋈ n2)
      → Gather (parallel workers)
        → Hash Join (lineitem ⋈ supplier)
          → Parallel Hash Join (lineitem ⋈ part)
            → Parallel Hash Join (lineitem ⋈ orders)      ← orders integrated here
              → Parallel Seq Scan on orders o              ← date filter applied here
```

```
→ Parallel Hash Semi Join (orders ⋈ customer WITHIN natioN/region CHECK)
```

The planner merges `orders_secure` completely into the main join, applies `o_orderdate BETWEEN 1995 AND 1996` early on the orders scan (reducing 1.5M orders to ~152k), and uses a Hash Semi Join to push the ASIA region filter alongside the other join conditions. Everything happens in one coordinated parallel pass.

Factor 3 — Date Predicate Pushdown

RLS plan:

```
Seq Scan on public.orders  
Filter: (o_orderstatus... AND o_orderdate BETWEEN ...)  
→ But the RLS EXISTS check runs FIRST for every row  
→ actual rows returned: 91,707 (after EXISTS check)
```

The `o_orderdate` range filter should be cheap and highly selective, but it cannot be used to skip the RLS EXISTS check. PostgreSQL must evaluate EXISTS for every row before it can apply the date filter.

View plan:

```
Parallel Seq Scan on public.orders o  
Filter: (o.o_orderdate >= '1995-01-01' AND o.o_orderdate <=  
'1996-12-31')  
actual time=0.686..94ms rows=152,421
```

The planner applies the date range directly to the raw orders scan, reducing 1.5M rows to 152k, then feeds those through the ASIA join check. Highly selective filters work as intended.

Factor 4 — Nested Loop vs Hash Join for Orders

RLS:

```

Nested Loop (cost=42.17..8,435,975,046)
  → Seq Scan on orders (cost=8,394,198,508 ← security barrier cost)
  → Materialize → Hash Join (customer filter) ←re-evaluated per orders row
    actual time: 85ms..246,533ms for this subtree alone

```

The security barrier forces a Nested Loop where orders is the outer table and the customer/nation/region correlated check is the inner. With 1.5M order rows as the outer side, the inner customer join executes 1.5M times. The `Materialize` node caches the inner result across loops, but it still processes the outer side row-by-row.

View:

```

Parallel Hash Join (orders ⋈ customer with nation/region)
  actual time: 11ms..132ms

```

One hash build + one scan. The difference is not measured in seconds, it's `246,533ms vs 132ms` for effectively the same logical operation.

Factor 5 — Parallelism Utilization

RLS: Parallel workers are used for lineitem and part scans, but the orders Nested Loop runs **single-threaded** because the security barrier prevents parallel execution of the correlated subquery chain.

View: The entire inner join subtree including orders, customer, nation, region runs with **parallel workers**. The plan shows `Gather` collecting from workers that each handle ~50k customers and ~152k orders in parallel.

Summary Table — Q8

Aspect	RLS	View
Orders scan cost	8,394,198,508	35,383
Orders actual time	1,258ms	94ms

Aspect	RLS	View
Customer check style	Correlated EXISTS per order row	One Hash Semi Join
Date filter applied	After EXISTS check	Before any join
Join strategy for orders	Nested Loop (barrier-forced)	Parallel Hash Join
Parallelism	Partial (lineitem/part only)	Full (all tables)
Execution Time	745,310ms	825ms

The one-line summary: The RLS barrier converts a simple date-filtered join into 1.5 million individual EXISTS correlated subquery evaluations. The view has no barrier, so the planner produces a plan virtually identical to the baseline — hence 825ms vs 745 seconds.

Q8 is where the real story is. The RLS policy filters orders by checking whether each order's customer belongs to the ASIA region – a sensible filter – but because it's an RLS security barrier, PostgreSQL is legally forbidden from reordering or merging that check with the rest of the query. So instead of doing a smart Hash Join once, it has to run the customer → nation → region EXISTS check individually for every single one of the 1.5 million orders rows before it can even apply the date filter or join anything else – that one decision alone pushes the orders scan cost from ~35,000 to ~8.4 billion. The view has no such restriction, so the planner sees the whole query as one unified join tree, applies the date filter first (cutting 1.5M orders down to 152k), then does a single parallel Hash Semi Join to check the ASIA region condition – exactly what a good query planner would naturally do. Same logical filter, but one approach takes 825ms and the other takes 12 minutes, purely because of how PostgreSQL's security barrier prevents the planner from doing its job.